Report 1

Andrei Petrea
Faculty of Automatic Control and Computers
National University of Science and Technology POLITEHNICA Bucharest

June 13, 2026

Abstract

This report examines container image minimization in the context of modern day use-cases. It provides an overview of the state-of-the-art tools and techniques for minimizing container images, as well as a detailed description of my work in developing a new tool for this purpose. The report concludes with a comparison of my tool with existing solutions and a discussion of potential future work in this area.

1 Introduction

It has been almost a decade since the famous *Attention Is All You Need* [5] paper was published, which began the era of artificial intelligence (AI) development, culminating in the creation of large language models (LLMs) such as ChatGPT.

In the past few years, LLMs have gradually transitioned from simple chatbots into highly sophisticated tools that can perform a wide range of tasks. This transition is characterized by a migration away from simple prompt-and-response dynamics toward continuous, goal-oriented autonomy. Modern agentic frameworks leverage the underlying model not just for linguistic fluency, but as a dynamic engine for systemic interaction—capable of maintaining context over extended horizons, utilizing specialized tools, and executing complex, multi-stage workflows within broader software architectures.

As such, it is natural to explore how LLMs can be utilized to build minimal container images. In this context, I created an agentic workflow where LLMs are leveraged to fuzz the application and in order to find as many execution paths in order to more accurately determine the minimal set of dependencies required to run and interact with the application.

2 Architecture

What are AI agents?

An AI agent is an autonomous system that performs tasks by designing and executing workflows with available tools. They overcome the limitations of traditional LLMs by being augmented with memory, tool use, and the ability to plan and execute multi-step processes. This allows them to handle complex tasks that require reasoning, decision-making, and interaction with external systems [2].

The typical AI agent flow consists of the following three components:

- **Goal initialization & planning**: Human-defined goals and rules guide the agent to decompose tasks into subtasks and create a plan.
- **Reasoning with tools**: The agent calls external datasets, APIs, or other agents to fill information gaps, then updates its knowledge and refines its plan.
- **Learning & reflection**: Feedback from humans, other agents, or HITL(Human-in-the-Loop) interactions is stored in memory, enabling iterative improvement and personalization.

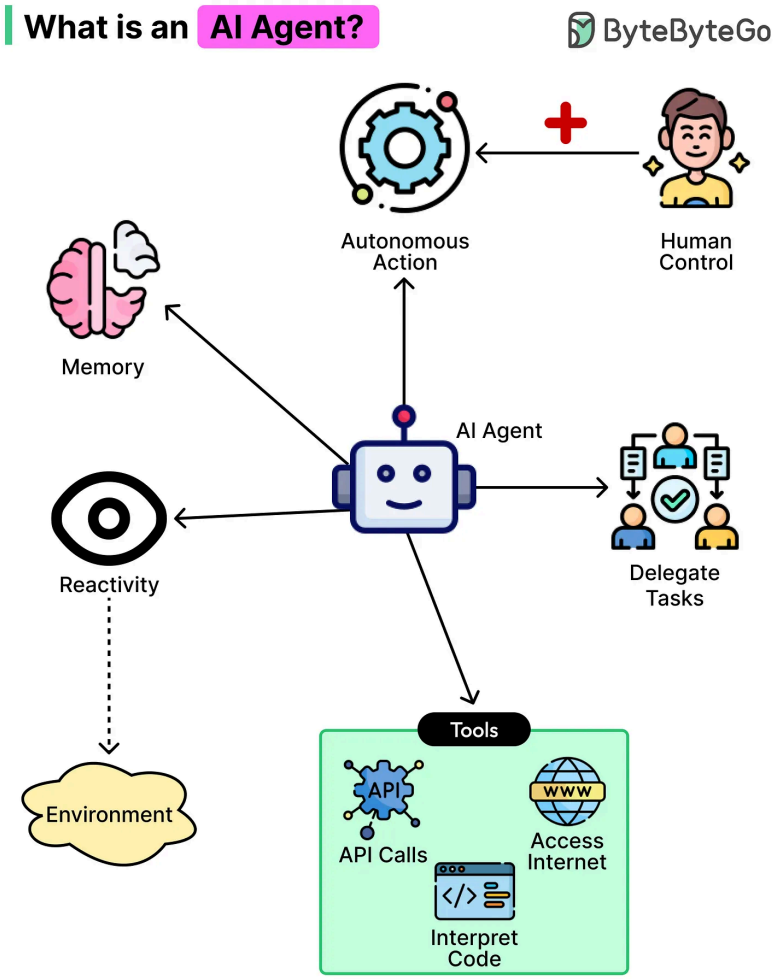

Figure 1: AI agents architecture

Using agents to minimize applications

Owing to their above mentioned capabilities, I identified two ways they can be used in my project:

- **AI Fuzzing Agent**: An agent that fuzzes the application to trigger state changes in order to identify hidden dependencies
- **AI Testing Agent**: An agent that tests the minimize application to ensure that it runs the same as the original application and that it is free of bugs

For this report, I will focus on the *AI Fuzzing Agent*, leaving the testing one for future work.

As you can recall from the previous report [3], my application tries to build the minimal container image in three steps: a static analysis using ldd, a dynamic analysis using strace and a brute-force binary search in case the first two failed. In theory, the final binary stage is sufficient since, given enough time, it will eventually find the minimal set. However, in practice, it is very resource-intensive and the final result is not much smaller than the original one (multiple binary-searches one after the other will asymptotically approach the minimal set).

As such, the best way to improve the performance of the project is to improve the dynamic analysis stage, which is where the AI Fuzzing Agent comes in. As the name implies, the agent will act as a fuzzer, an automated testing technique that injects input into a program to trigger state changes [1].

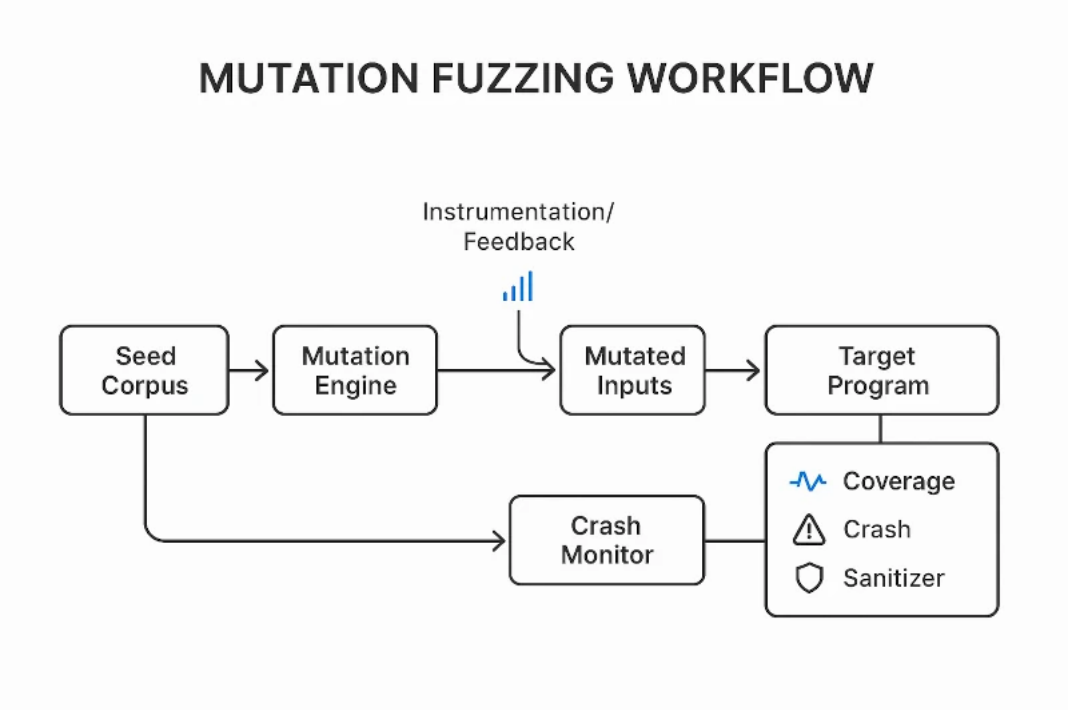

Figure 2: A typical fuzzing flow

The goal of the agent is to send input to the application whilst it's being traced by strace, in order to trigger as many state changes to uncover any hidden dependencies that the app may need in order to run.

3 Implementation

The implementation for the agent is written in Go in order to have a much simpler integration with the larger project, which is also written in Go. To facilitate the development of the agent, I used the Google Gen AI SDK, which provides a simple interface for interacting with their Gemini models. However, I made sure that using any provider's models is a simple plug-and-play operation.

Foremost, I change the cli interface, adding the following flags:

- `--provider` - AI provider to use for analysis (e.g., 'google')
- `--model` - AI model to use for analysis (e.g., 'gemini-3.5-flash')
- `--token` - API token for the AI service

These 3 flags are optional, and if not provided, it will use the standard dynamic analysis as before.

In order to provide an agnostic interface for the AI models, I create an *Agent* interface with two methods

```
1 type Agent interface {
2         GenerateAction(ctx context.Context, prompt string) (
             string, error)
3         RegisterTools(tools []tools.Tool)
4 }
```

Listing 1: Agent interface

The `GenerateAction` method takes a prompt and returns the LLMs reasoning process alongside any action it took. The `RegisterTools` method is used to register the tools that the agent can use to perform actions.

In our context, a *tool* is an object that implements the `Tool`.

```
1 type Tool struct {
2         Name        string
3         Description string
4         InputSchema map[string]interface{}
5         Execute     func(containerName string, input map[
             string]interface{}) (string, error)
6 }
```

Listing 2: Tool interface

It provides a function with how to call it, plus a description and an input schema, which is used by the LLM to understand how to use it. For this agent, I have created 3 tools

- **executeCommand**: Execute a shell command inside the target container and capture stdout/stderr
- **writeToStdin**: Write raw string payloads directly into the standard input (stdin) of the main running container process

- **sendRawSocket**: Open a raw network socket (TCP or UDP) to a specific port on the container and transmit arbitrary data bytes.

These two interfaces provide our most basic abstractions in order to build our fuzzing agent. Let us see how to use them to integrate the Google Gemini model into our agent.

In order to use any provider, we simply create a new struct that implements the `Agent` interface. For the Google Gemini model, I created the `GoogleAIAgent` struct.

```
import (
        "google.golang.org/genai"
)

type GoogleAIAgent struct {
        client       *genai.Client
        model        string
        systemPrompt string
        tools        []tools.Tool
}
```

Listing 3: Google Ai Agent

We then provide a definition for the two methods of the `Agent` interface and a constructor and we are done. A key point to note is that we need to provide the model with a **system prompt**, which is a string that describes the context in which the model is operating, the tools it has at its disposal and how to use them. This is a crucial step in order to have a good performance from the model, as it needs to understand the environment and the tools in order to use them effectively [4].

To instantiate the agent, we use a *factory* function that takes the provider, model and token and returns the corresponding agent.

```
var agents = map[string]func(context.Context, string, string)
    Agent{
        "google": NewGoogleAIAgent,
}

func AgentFactory(ctx context.Context, maker string, model
   string, token string) Agent {
        if agent, exists := agents[maker]; exists {
                return agent(ctx, model, token)
        }
        return nil
}
```

Listing 4: Agent Factory

4 Results and Further Work

Results

I run my new implementation on the same dataset as with the original one and I have found some interesting results.

Broadly speaking, the new dynamic analysis was able to find more dependencies than the standard one, but not that many more and only few application benefited from it, mainly the ones that had the source code available on the container itself. Additionally, the AI was prone to send data to the container that caused it to crash, skipping the rest of the analysis and going to the binary search. I altered the system prompt to try to mitigate this but it was still an issue. I also found that it was consuming tokens at a much higher rate than I expected, especially when the source code was available. Another issue was that I was the AI began hallucinating after a while, sending data that was not relevant to the analysis.

Further Work

Based on the results I obtained, I think that the most important thing to do is to fix the issues I mentioned above, especially fixing the crashes.

For the future, I have some ideas, including:

- **Support for more models**: I want to add support for more providers, especially OpenAI and Anthropic
- **More dynamic analysis traces**: I want to add more probes to trace the application, including *ftrace*, *fanotify* etc.
- **Compose support**: I want to add a wrapper for docker compose, to minimize a whole stack at once, instead of doing it for each image separately.

References

[1] BlackDuck. Fuzz testing. https://www.blackduck.com/glossary/what-is-fuzz-testing.html.

[2] A. Gutowska. Ai agents defined. https://www.ibm.com/think/topics/ai-agents.

[3] A. Petrea. Report 1, 2026.

[4] Tetrate.io. System prompts: Design patterns and best practices. https://tetrate.io/learn/ai/system-prompts-guide.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. https://arxiv.org/abs/1706.03762, 2017.